

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	K.Hemasree	Reg. No.:	192411156
Section / Batch:	Slot c	Date:	18/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I K.Hemasree(192411156)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				

Total Marks (100):

Faculty In-charge

Course Coordinator

Head of Department

Name:

Name:

Name:

Signature: _____

Signature: _____

Signature: _____

Date: _____

Date: _____

Date: _____

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

KEERTHIPATI HEMA SREE
192411026RC

Language: Python C C++ Java

QUESTIONS

Q1

Cryptocurrency Price Tracker (Quick Sort) A cryptocurrency platform tracks and sorts prices dynamically for thousands of coins

100 marks

1/1 answered

Q1 Cryptocurrency Price Tracker (Quick Sort) A cryptocurrency platform tracks and sorts prices dynamically for thousands of coins 100 marks

Analyze how Quick Sort can be applied for fast sorting. Identify issues such as pivot selection and fluctuating data. Apply divide and conquer concepts to explain its working. Analyze performance and propose optimizations with proper justification.

Your Code

```
1 import random
2
3 def get_median_of_three(arr, low, high):
4     """
5     Optimization: Selects the median of the first, middle, and last
6     elements to avoid O(n^2) worst-case performance on sorted data.
7     """
8     mid = (low + high) // 2
9     # Sort the three elements to find the median
10    if arr[low] > arr[mid]:
11        arr[low], arr[mid] = arr[mid], arr[low]
12    if arr[low] > arr[high]:
13        arr[low], arr[high] = arr[high], arr[low]
14    if arr[mid] > arr[high]:
15        arr[mid], arr[high] = arr[high], arr[mid]
16
17    # Place median at the end to act as pivot
18    arr[mid], arr[high] = arr[high], arr[mid]
19    return arr[high]
20
21 def partition(arr, low, high):
22     """
23     Partitions the array around the chosen pivot.
24     """
25     # Apply Median-of-Three optimization
26     pivot = get_median_of_three(arr, low, high)
```

Input

stdin input

Output

```
--- Cryptocurrency Price Tracker
---
Original Prices: (45200.5,
11200.78, 35000.0, 0.45, 45000.2,
2200.4, 150.1, 42000.0)
Sorted Prices: [0.45, 11200.78, 150.1, 2200.4, 35000.0, 42000.0, 45000.2, 45200.5]
```